

Modelo negocio Fintech

Continuamos con el proyecto Integrador que consistía en una aplicación codificada en JS para una pa FINTECH. Habíamos generado la clase Cliente. Ahora necesitaríamos en esta tercera etapa generar la clase Cuenta y el resto de las clases necesarias. Vamos a generar el modelo de negocio aplicando las buenas prácticas de programación orientada a objetos (principio SRP, documentar las clases correctamente para que sean de fácil mantención) y mostrando las relaciones entre clases. Luego generamos las clases asociadas al modelo asegurando encapsulamiento y validación de constructores, setters y getters. Finalmente devolveremos las clases Test que incluyan las pruebas de caja negra de todos los métodos de las clases principales. Por ahora no devolvemos ninguna clase "App" solo el modelo del negocio y las clases codificadas.

Etapla 3 — Modelo de negocio de la Fintech

En esta etapa incorporamos las cuentas bancarias, los movimientos y las transferencias. Se mantiene la clase `Cliente`, ajustándola para que todo el modelo utilice las mismas reglas de encapsulamiento y validación.

La propuesta evita relaciones bidireccionales innecesarias:

- `Cuenta` conoce a su `Cliente`.
- `Cliente` no almacena una colección de cuentas.
- Las cuentas de un cliente podrán obtenerse posteriormente mediante un repositorio o servicio.
- `Cuenta` sí almacena sus movimientos, porque estos forman parte de su historial.
- `TransferenciaService` coordina operaciones entre dos cuentas.

Esto evita mantener información duplicada en `Cliente` y `Cuenta`.

1. Clases del modelo

Cliente

Representa a una persona registrada en la fintech.

Responsabilidades:

- Mantener los datos personales.
- Validar nombre, apellido, DNI y correo.
- Validar credenciales.
- Permitir cambiar datos personales y contraseña.

Cuenta

Representa una cuenta bancaria perteneciente a un cliente.

Responsabilidades:

- Mantener su código, saldo, titular y estado.
- Realizar depósitos.
- Realizar retiros.
- Registrar movimientos.
- Bloquear, desbloquear y cerrar la cuenta.

Movimiento

Representa una operación registrada en una cuenta.

Responsabilidades:

- Mantener la información histórica de una operación.
- Informar tipo, monto, fecha, descripción y saldo posterior.

Los movimientos son inmutables: una vez creados, no deberían modificarse.

TransferenciaService

Coordina una transferencia entre dos cuentas.

Responsabilidades:

- Validar las cuentas involucradas.
- Debitar el dinero de la cuenta de origen.
- Acreditarlo en la cuenta de destino.
- Generar los movimientos correspondientes.

La transferencia no se incorpora directamente dentro de **Cuenta** porque involucra dos objetos diferentes.

TipoMovimiento

Define los tipos de operaciones permitidos.

EstadoCuenta

Define los posibles estados de una cuenta.

2. Relaciones entre clases

- Un **Cliente** puede estar asociado con cero o muchas cuentas.
- Cada **Cuenta** pertenece a exactamente un **Cliente**.
- Una **Cuenta** contiene cero o muchos movimientos.
- Cada **Movimiento** pertenece al historial de una sola cuenta.
- **TransferenciaService** depende de **Cuenta**.

La relación entre **Cuenta** y **Movimiento** se modela como **composición**, porque un movimiento no tiene sentido fuera del historial de una cuenta.

La relación entre **Cliente** y **Cuenta** se modela como una **asociación**, navegable desde **Cuenta** hacia **Cliente**.

3. Diagrama UML en Mermaid

Mermaid

```
classDiagram
```

```
class Cliente {  
    -String id  
    -String nombre  
    -String apellido  
    -String dni  
    -String email  
    -String password  
    +Cliente(id, nombre, apellido, dni, email, password)  
    +getId() String  
    +getNombre() String  
    +setNombre(nombre) void  
    +getApellido() String  
    +setApellido(apellido) void  
    +getDni() String  
    +setDni(dni) void  
    +getEmail() String  
}
```

4. Estructura de archivos

```
fintech/  
|
```

```
├─ src/
│   └─ model/
│       ├── Cliente.js
│       ├── Cuenta.js
│       ├── Movimiento.js
│       ├── TipoMovimiento.js
│       └─ EstadoCuenta.js
│   └─ service/
│       └─ TransferenciaService.js
├─ test/
│   ├── Cliente.test.js
│   ├── Cuenta.test.js
│   ├── Movimiento.test.js
│   └─ TransferenciaService.test.js
└─ package.json
```

5. Enumeración TipoMovimiento

src/model/TipoMovimiento.js

JavaScript

```
/**
 * Tipos de movimientos admitidos por el sistema.
 *
 * El objeto se congela para impedir que los tipos sean
 * modificados durante la ejecución de la aplicación.
 */
export const TipoMovimiento = Object.freeze({
  DEPOSITO: "DEPOSITO",
  RETIRO: "RETIRO",
  TRANSFERENCIA_ENVIADA: "TRANSFERENCIA_ENVIADA",
  TRANSFERENCIA_RECIBIDA: "TRANSFERENCIA_RECIBIDA"
});
```

6. Enumeración EstadoCuenta

src/model/EstadoCuenta.js

JavaScript

```
/**
 * Estados posibles de una cuenta bancaria.
 */
```

```
export const EstadoCuenta = Object.freeze({
  ACTIVA: "ACTIVA",
  BLOQUEADA: "BLOQUEADA",
  CERRADA: "CERRADA"
});
```

7. Clase Cliente

src/model/Cliente.js

</> JavaScript

```
/**
 * Representa a un cliente registrado en la fintech.
 *
 * La clase es responsable de mantener y validar los datos
 * personales y las credenciales del cliente.
 */
export class Cliente {

  #id;
  #nombre;
  #apellido;
  #dni;
  #email;
  #password;

  /**
   * Crea un nuevo cliente.
   *
   * @param {string} id Identificador único.
   * @param {string} nombre Nombre del cliente.
   * @param {string} apellido Apellido del cliente.
   * @param {string} dni Documento del cliente.
   * @param {string} email Correo electrónico.
   * @param {string} password Contraseña.
   */
  constructor(id, nombre, apellido, dni, email, password) {
    this.#validarId(id);
    this.#validarPassword(password);

    this.#id = id.trim();
    this.nombre = nombre;
    this.apellido = apellido;
    this.dni = dni;
    this.email = email;
    this.#password = password;
  }

  get id() {
```

```
        return this.#id;
    }

    get nombre() {
        return this.#nombre;
    }

    set nombre(nombre) {
        this.#validarTexto(nombre, "El nombre");
        this.#nombre = nombre.trim();
    }

    get apellido() {
        return this.#apellido;
    }

    set apellido(apellido) {
        this.#validarTexto(apellido, "El apellido");
        this.#apellido = apellido.trim();
    }

    get dni() {
        return this.#dni;
    }

    set dni(dni) {
        if (typeof dni !== "string" || !/^d{7,8}$/.test(dni.trim())) {
            throw new Error(
                "El DNI debe contener entre 7 y 8 dígitos."
            );
        }

        this.#dni = dni.trim();
    }

    get email() {
        return this.#email;
    }

    set email(email) {
        if (typeof email !== "string") {
            throw new Error("El correo electrónico debe ser una cadena.");
        }

        const emailNormalizado = email.trim().toLowerCase();
        const expresionEmail = /^[^s@]+@[^s@]+\.[^s@]+$/;

        if (!expresionEmail.test(emailNormalizado)) {
            throw new Error("El correo electrónico no tiene un formato válido.");
        }

        this.#email = emailNormalizado;
    }
}
```

```
/**
 * Devuelve el nombre completo del cliente.
 *
 * @returns {string}
 */
getNombreCompleto() {
    return `${this.#nombre} ${this.#apellido}`;
}

/**
 * Comprueba si una contraseña coincide con la registrada.
 *
 * @param {string} password Contraseña a verificar.
 * @returns {boolean}
 */
autenticar(password) {
    return this.#password === password;
}

/**
 * Cambia la contraseña del cliente.
 *
 * @param {string} passwordActual Contraseña actualmente registrada.
 * @param {string} passwordNueva Nueva contraseña.
 */
cambiarPassword(passwordActual, passwordNueva) {
    if (!this.autenticar(passwordActual)) {
        throw new Error("La contraseña actual es incorrecta.");
    }

    this.#validarPassword(passwordNueva);

    if (passwordActual === passwordNueva) {
        throw new Error(
            "La nueva contraseña debe ser diferente de la actual."
        );
    }

    this.#password = passwordNueva;
}

#validarId(id) {
    this.#validarTexto(id, "El identificador");
}

#validarPassword(password) {
    if (typeof password !== "string" || password.length < 6) {
        throw new Error(
            "La contraseña debe contener al menos 6 caracteres."
        );
    }
}
```

```
#validarTexto(valor, nombreCampo) {  
  if (typeof valor !== "string" || valor.trim() === "") {  
    throw new Error(`${nombreCampo} es obligatorio.`);  
  }  
}  
}
```

La contraseña no tiene getter público. De esta manera, ningún objeto externo puede consultar directamente la contraseña almacenada.

En una aplicación real, la contraseña debería guardarse utilizando un mecanismo de hash y nunca como texto plano. En esta etapa se mantiene el tratamiento sencillo para concentrarnos en el modelo orientado a objetos.

8. Clase Movimiento

src/model/Movimiento.js

JavaScript

```
import { TipoMovimiento } from "../TipoMovimiento.js";  
  
/**  
 * Representa un movimiento registrado en una cuenta.  
 *  
 * Los movimientos son objetos inmutables. Una vez creados,  
 * sus datos no pueden modificarse mediante setters.  
 */  
export class Movimiento {  
  
  #id;  
  #tipo;  
  #monto;  
  #fecha;  
  #descripcion;  
  #saldoPosterior;  
  
  /**  
   * @param {string} id Identificador único del movimiento.  
   * @param {string} tipo Tipo de movimiento.  
   * @param {number} monto Monto involucrado.  
   * @param {Date} fecha Fecha de la operación.  
   * @param {string} descripcion Descripción de la operación.  
   * @param {number} saldoPosterior Saldo resultante.  
   */  
  constructor(  
    id,  
    tipo,
```



```
        monto,
        fecha,
        descripcion,
        saldoPosterior
    ) {
        this.#validarId(id);
        this.#validarTipo(tipo);
        this.#validarMonto(monto);
        this.#validarFecha(fecha);
        this.#validarDescripcion(descripcion);
        this.#validarSaldoPosterior(saldoPosterior);

        this.#id = id.trim();
        this.#tipo = tipo;
        this.#monto = monto;
        this.#fecha = new Date(fecha.getTime());
        this.#descripcion = descripcion.trim();
        this.#saldoPosterior = saldoPosterior;
    }

    get id() {
        return this.#id;
    }

    get tipo() {
        return this.#tipo;
    }

    get monto() {
        return this.#monto;
    }

    get fecha() {
        /*
         * Se devuelve una copia para evitar que un objeto externo
         * modifique la fecha interna del movimiento.
         */
        return new Date(this.#fecha.getTime());
    }

    get descripcion() {
        return this.#descripcion;
    }

    get saldoPosterior() {
        return this.#saldoPosterior;
    }

    #validarId(id) {
        if (typeof id !== "string" || id.trim() === "") {
            throw new Error("El identificador del movimiento es obligatorio.");
        }
    }
}
```

```
#validarTipo(tipo) {
  const tiposValidos = Object.values(TipoMovimiento);

  if (!tiposValidos.includes(tipo)) {
    throw new Error("El tipo de movimiento no es válido.");
  }
}

#validarMonto(monto) {
  if (
    typeof monto !== "number" ||
    !Number.isFinite(monto) ||
    monto <= 0
  ) {
    throw new Error(
      "El monto del movimiento debe ser mayor que cero."
    );
  }
}

#validarFecha(fecha) {
  if (
    !(fecha instanceof Date) ||
    Number.isNaN(fecha.getTime())
  ) {
    throw new Error("La fecha del movimiento no es válida.");
  }
}

#validarDescripcion(descripcion) {
  if (
    typeof descripcion !== "string" ||
    descripcion.trim() === ""
  ) {
    throw new Error(
      "La descripción del movimiento es obligatoria."
    );
  }
}

#validarSaldoPosterior(saldoPosterior) {
  if (
    typeof saldoPosterior !== "number" ||
    !Number.isFinite(saldoPosterior) ||
    saldoPosterior < 0
  ) {
    throw new Error(
      "El saldo posterior no puede ser negativo."
    );
  }
}
```

```
}  
}
```

9. Clase Cuenta

src/model/Cuenta.js

</> JavaScript

```
import { Cliente } from "../Cliente.js";  
import { Movimiento } from "../Movimiento.js";  
import { TipoMovimiento } from "../TipoMovimiento.js";  
import { EstadoCuenta } from "../EstadoCuenta.js";  
  
/**  
 * Representa una cuenta bancaria de La fintech.  
 *  
 * La cuenta administra:  
 *  
 * - Su saldo.  
 * - Su estado.  
 * - Los depósitos.  
 * - Los retiros.  
 * - Su historial de movimientos.  
 */  
export class Cuenta {  
  
    #codigo;  
    #titular;  
    #saldo;  
    #estado;  
    #movimientos;  
    #secuenciaMovimientos;  
  
    /**  
     * Crea una cuenta bancaria.  
     *  
     * @param {string} codigo Código único de la cuenta.  
     * @param {Cliente} titular Cliente titular.  
     * @param {number} saldoInicial Saldo inicial.  
     */  
    constructor(codigo, titular, saldoInicial = 0) {  
        this.#validarCodigo(codigo);  
        this.#validarTitular(titular);  
        this.#validarSaldoInicial(saldoInicial);  
  
        this.#codigo = codigo.trim();  
        this.#titular = titular;  
        this.#saldo = saldoInicial;  
        this.#estado = EstadoCuenta.ACTIVA;
```

```
this.#movimientos = [];  
this.#secuenciaMovimientos = 0;  
  
if (saldoInicial > 0) {  
    this.#registrarMovimiento(  
        TipoMovimiento.DEPOSITO,  
        saldoInicial,  
        "Saldo inicial",  
        new Date()  
    );  
}  
}  
  
get codigo() {  
    return this.#codigo;  
}  
  
get titular() {  
    return this.#titular;  
}  
  
get saldo() {  
    return this.#saldo;  
}  
  
get estado() {  
    return this.#estado;  
}  
  
/**  
 * Devuelve una copia del arreglo de movimientos.  
 *  
 * De esta manera, el arreglo interno no puede modificarse  
 * desde el exterior.  
 *  
 * @returns {Movimiento[]}  
 */  
get movimientos() {  
    return [...this.#movimientos];  
}  
  
/**  
 * Indica si la cuenta se encuentra activa.  
 *  
 * @returns {boolean}  
 */  
estaActiva() {  
    return this.#estado === EstadoCuenta.ACTIVA;  
}  
  
/**  
 * Deposita dinero en la cuenta.  
 *
```

```

    * @param {number} monto Monto a depositar.
    * @param {string} descripcion Descripción de La operación.
    * @param {Date} fecha Fecha de La operación.
    * @param {string} tipo Tipo de movimiento.
    * @returns {Movimiento}
    */
    depositar(
        monto,
        descripcion = "Depósito",
        fecha = new Date(),
        tipo = TipoMovimiento.DEPOSITO
    ) {
        this.#validarCuentaOperativa();
        this.#validarMonto(monto);
        this.#validarFecha(fecha);
        this.#validarDescripcion(descripcion);

        const tiposPermitidos = [
            TipoMovimiento.DEPOSITO,
            TipoMovimiento.TRANSFERENCIA_RECIBIDA
        ];

        if (!tiposPermitidos.includes(tipo)) {
            throw new Error(
                "El tipo indicado no corresponde a una operación de crédito."
            );
        }

        this.#saldo += monto;

        return this.#registrarMovimiento(
            tipo,
            monto,
            descripcion,
            fecha
        );
    }

    /**
     * Retira dinero de La cuenta.
     *
     * @param {number} monto Monto a retirar.
     * @param {string} descripcion Descripción de La operación.
     * @param {Date} fecha Fecha de La operación.
     * @param {string} tipo Tipo de movimiento.
     * @returns {Movimiento}
     */
    retirar(
        monto,
        descripcion = "Retiro",
        fecha = new Date(),
        tipo = TipoMovimiento.RETIRO
    ) {

```

```
this.#validarCuentaOperativa();
this.#validarMonto(monto);
this.#validarFecha(fecha);
this.#validarDescripcion(descripcion);

const tiposPermitidos = [
  TipoMovimiento.RETIRO,
  TipoMovimiento.TRANSFERENCIA_ENVIADA
];

if (!tiposPermitidos.includes(tipo)) {
  throw new Error(
    "El tipo indicado no corresponde a una operación de débito."
  );
}

if (monto > this.#saldo) {
  throw new Error("Saldo insuficiente.");
}

this.#saldo -= monto;

return this.#registrarMovimiento(
  tipo,
  monto,
  descripcion,
  fecha
);
}

/**
 * Bloquea temporalmente la cuenta.
 */
bloquear() {
  if (this.#estado === EstadoCuenta.CERRADA) {
    throw new Error("No se puede bloquear una cuenta cerrada.");
  }

  if (this.#estado === EstadoCuenta.BLOQUEADA) {
    throw new Error("La cuenta ya se encuentra bloqueada.");
  }

  this.#estado = EstadoCuenta.BLOQUEADA;
}

/**
 * Desbloquea una cuenta bloqueada.
 */
desbloquear() {
  if (this.#estado === EstadoCuenta.CERRADA) {
    throw new Error("No se puede desbloquear una cuenta cerrada.");
  }
}
```

```
        if (this.#estado !== EstadoCuenta.BLOQUEADA) {
            throw new Error("La cuenta no se encuentra bloqueada.");
        }

        this.#estado = EstadoCuenta.ACTIVA;
    }

    /**
     * Cierra definitivamente la cuenta.
     *
     * Para cerrar una cuenta, el saldo debe ser cero.
     */
    cerrar() {
        if (this.#estado === EstadoCuenta.CERRADA) {
            throw new Error("La cuenta ya se encuentra cerrada.");
        }

        if (this.#saldo !== 0) {
            throw new Error(
                "La cuenta solo puede cerrarse cuando su saldo es cero."
            );
        }

        this.#estado = EstadoCuenta.CERRADA;
    }

    #registrarMovimiento(tipo, monto, descripcion, fecha) {
        this.#secuenciaMovimientos++;

        const idMovimiento =
            `${this.#codigo}-MOV-${this.#secuenciaMovimientos}`;

        const movimiento = new Movimiento(
            idMovimiento,
            tipo,
            monto,
            fecha,
            descripcion,
            this.#saldo
        );

        this.#movimientos.push(movimiento);

        return movimiento;
    }

    #validarCodigo(codigo) {
        if (typeof codigo !== "string" || codigo.trim() === "") {
            throw new Error("El código de la cuenta es obligatorio.");
        }
    }

    #validarTitular(titular) {
```

```
        if (!(titular instanceof Cliente)) {
            throw new Error(
                "El titular de la cuenta debe ser un objeto Cliente."
            );
        }
    }

    #validarSaldoInicial(saldoInicial) {
        if (
            typeof saldoInicial !== "number" ||
            !Number.isFinite(saldoInicial) ||
            saldoInicial < 0
        ) {
            throw new Error(
                "El saldo inicial debe ser un número igual o mayor que cero."
            );
        }
    }

    #validarMonto(monto) {
        if (
            typeof monto !== "number" ||
            !Number.isFinite(monto) ||
            monto <= 0
        ) {
            throw new Error("El monto debe ser mayor que cero.");
        }
    }

    #validarFecha(fecha) {
        if (
            !(fecha instanceof Date) ||
            Number.isNaN(fecha.getTime())
        ) {
            throw new Error("La fecha de la operación no es válida.");
        }
    }

    #validarDescripcion(descripcion) {
        if (
            typeof descripcion !== "string" ||
            descripcion.trim() === ""
        ) {
            throw new Error(
                "La descripción de la operación es obligatoria."
            );
        }
    }

    #validarCuentaOperativa() {
        if (this.#estado === EstadoCuenta.BLOQUEADA) {
            throw new Error("La cuenta se encuentra bloqueada.");
        }
    }
}
```



```
        if (this.#estado === EstadoCuenta.CERRADA) {  
            throw new Error("La cuenta se encuentra cerrada.");  
        }  
    }  
}
```

10. Servicio de transferencias

src/service/TransferenciaService.js

</> JavaScript

```
import { Cuenta } from "../model/Cuenta.js";  
import { EstadoCuenta } from "../model/EstadoCuenta.js";  
import { TipoMovimiento } from "../model/TipoMovimiento.js";  
  
/**  
 * Servicio encargado de coordinar transferencias entre cuentas.  
 *  
 * La transferencia se separa de la clase Cuenta porque involucra  
 * simultáneamente a una cuenta de origen y otra de destino.  
 */  
export class TransferenciaService {  
  
    /**  
     * Transfiere dinero entre dos cuentas.  
     *  
     * @param {Cuenta} cuentaOrigen Cuenta que envía el dinero.  
     * @param {Cuenta} cuentaDestino Cuenta que recibe el dinero.  
     * @param {number} monto Monto a transferir.  
     * @param {string} descripcion Descripción de la transferencia.  
     * @param {Date} fecha Fecha de la transferencia.  
     * @returns {{movimientoOrigen: Object, movimientoDestino: Object}}  
     */  
    transferir(  
        cuentaOrigen,  
        cuentaDestino,  
        monto,  
        descripcion = "Transferencia",  
        fecha = new Date()  
    ) {  
        this.#validarCuenta(cuentaOrigen, "origen");  
        this.#validarCuenta(cuentaDestino, "destino");  
        this.#validarCuentasDiferentes(cuentaOrigen, cuentaDestino);  
        this.#validarMonto(monto);  
        this.#validarDescripcion(descripcion);  
        this.#validarFecha(fecha);  
        this.#validarEstado(cuentaOrigen, "origen");  
        this.#validarEstado(cuentaDestino, "destino");  
    }  
}
```

```
if (cuentaOrigen.saldo < monto) {
    throw new Error(
        "La cuenta de origen no tiene saldo suficiente."
    );
}

const descripcionOrigen =
    `${descripcion} a la cuenta ${cuentaDestino.codigo}`;

const descripcionDestino =
    `${descripcion} desde la cuenta ${cuentaOrigen.codigo}`;

const movimientoOrigen = cuentaOrigen.retirar(
    monto,
    descripcionOrigen,
    fecha,
    TipoMovimiento.TRANSFERENCIA_ENVIADA
);

const movimientoDestino = cuentaDestino.depositar(
    monto,
    descripcionDestino,
    fecha,
    TipoMovimiento.TRANSFERENCIA_RECIBIDA
);

return {
    movimientoOrigen,
    movimientoDestino
};
}

#validarCuenta(cuenta, nombre) {
    if (!(cuenta instanceof Cuenta)) {
        throw new Error(
            `La cuenta de ${nombre} debe ser un objeto Cuenta.`
        );
    }
}

#validarCuentasDiferentes(cuentaOrigen, cuentaDestino) {
    if (cuentaOrigen === cuentaDestino) {
        throw new Error(
            "La cuenta de origen y destino deben ser diferentes."
        );
    }

    if (cuentaOrigen.codigo === cuentaDestino.codigo) {
        throw new Error(
            "Las cuentas no pueden tener el mismo código."
        );
    }
}
```

```
}

#validarMonto(monto) {
  if (
    typeof monto !== "number" ||
    !Number.isFinite(monto) ||
    monto <= 0
  ) {
    throw new Error(
      "El monto de la transferencia debe ser mayor que cero."
    );
  }
}

#validarDescripcion(descripcion) {
  if (
    typeof descripcion !== "string" ||
    descripcion.trim() === ""
  ) {
    throw new Error(
      "La descripción de la transferencia es obligatoria."
    );
  }
}

#validarFecha(fecha) {
  if (
    !(fecha instanceof Date) ||
    Number.isNaN(fecha.getTime())
  ) {
    throw new Error(
      "La fecha de la transferencia no es válida."
    );
  }
}

#validarEstado(cuenta, nombre) {
  if (cuenta.estado !== EstadoCuenta.ACTIVA) {
    throw new Error(
      `La cuenta de ${nombre} debe encontrarse activa.`
    );
  }
}
}
```

11. Configuración de Jest

package.json

</> JSON

```
{
  "name": "fintech-poo",
  "version": "1.0.0",
  "description": "Modelo orientado a objetos para una aplicación Fintech",
  "type": "module",
  "scripts": {
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js",
    "test:watch": "node --experimental-vm-modules node_modules/jest/bin/jest.js --watch"
  },
  "devDependencies": {
    "jest": "^30.0.0"
  }
}
```

Instalación:

</> Bash

```
npm install
```

Ejecución de pruebas:

</> Bash

```
npm test
```

12. Pruebas de caja negra de Cliente

Estas pruebas utilizan únicamente la interfaz pública de la clase. No acceden a sus atributos privados.

test/Cliente.test.js

</> JavaScript

```
import { Cliente } from "../src/model/Cliente.js";

describe("Clase Cliente", () => {

  let cliente;

  beforeEach(() => {
    cliente = new Cliente(
      "CLI-001",
      "Ana",
      "Pérez",
      "30111222",
      "ANA@EMAIL.COM",
      "clave123"
    );
  });
});
```

```
    });  
});  
  
test("debe crear un cliente con datos válidos", () => {  
    expect(cliente.id).toBe("CLI-001");  
    expect(cliente.nombre).toBe("Ana");  
    expect(cliente.apellido).toBe("Pérez");  
    expect(cliente.dni).toBe("30111222");  
    expect(cliente.email).toBe("ana@email.com");  
});  
  
test("debe devolver el nombre completo", () => {  
    expect(cliente.getNombreCompleto()).toBe("Ana Pérez");  
});  
  
test("debe modificar el nombre", () => {  
    cliente.nombre = "María";  
  
    expect(cliente.nombre).toBe("María");  
    expect(cliente.getNombreCompleto()).toBe("María Pérez");  
});  
  
test("debe modificar el apellido", () => {  
    cliente.apellido = "Gómez";  
  
    expect(cliente.apellido).toBe("Gómez");  
});  
  
test("debe modificar el DNI con un valor válido", () => {  
    cliente.dni = "28123456";  
  
    expect(cliente.dni).toBe("28123456");  
});  
  
test("debe normalizar el correo electrónico", () => {  
    cliente.email = "NUEVO@EMAIL.COM";  
  
    expect(cliente.email).toBe("nuevo@email.com");  
});  
  
test("debe autenticar una contraseña correcta", () => {  
    expect(cliente.autenticar("clave123")).toBe(true);  
});  
  
test("debe rechazar una contraseña incorrecta", () => {  
    expect(cliente.autenticar("otraClave")).toBe(false);  
});  
  
test("debe cambiar la contraseña", () => {  
    cliente.cambiarPassword("clave123", "nuevaClave");  
  
    expect(cliente.autenticar("clave123")).toBe(false);  
    expect(cliente.autenticar("nuevaClave")).toBe(true);  
});
```

```
});

test("debe rechazar el cambio cuando la contraseña actual es incorrecta", () => {
  expect(() => {
    cliente.cambiarPassword("incorrecta", "nuevaClave");
  }).toThrow("La contraseña actual es incorrecta.");
});

test("debe rechazar una contraseña nueva demasiado corta", () => {
  expect(() => {
    cliente.cambiarPassword("clave123", "123");
  }).toThrow(
    "La contraseña debe contener al menos 6 caracteres."
  );
});

test("debe rechazar una contraseña igual a la actual", () => {
  expect(() => {
    cliente.cambiarPassword("clave123", "clave123");
  }).toThrow(
    "La nueva contraseña debe ser diferente de la actual."
  );
});

test("debe rechazar un identificador vacío", () => {
  expect(() => {
    new Cliente(
      "",
      "Ana",
      "Pérez",
      "30111222",
      "ana@email.com",
      "clave123"
    );
  }).toThrow("El identificador es obligatorio.");
});

test("debe rechazar un nombre vacío", () => {
  expect(() => {
    cliente.nombre = " ";
  }).toThrow("El nombre es obligatorio.");
});

test("debe rechazar un apellido vacío", () => {
  expect(() => {
    cliente.apellido = "";
  }).toThrow("El apellido es obligatorio.");
});

test("debe rechazar un DNI con letras", () => {
  expect(() => {
    cliente.dni = "30ABC222";
  }).toThrow(
```

```
        "El DNI debe contener entre 7 y 8 dígitos."
    );
});

test("debe rechazar un correo electrónico inválido", () => {
    expect(() => {
        cliente.email = "correo-invalido";
    }).toThrow(
        "El correo electrónico no tiene un formato válido."
    );
});
});
```

13. Pruebas de caja negra de Movimiento

test/Movimiento.test.js

</> JavaScript

```
import { Movimiento } from "../src/model/Movimiento.js";
import { TipoMovimiento } from "../src/model/TipoMovimiento.js";

describe("Clase Movimiento", () => {

    test("debe crear un movimiento válido", () => {
        const fecha = new Date("2026-07-28T15:00:00");

        const movimiento = new Movimiento(
            "CTA-001-MOV-1",
            TipoMovimiento.DEPOSITO,
            5000,
            fecha,
            "Depósito inicial",
            5000
        );

        expect(movimiento.id).toBe("CTA-001-MOV-1");
        expect(movimiento.tipo).toBe(TipoMovimiento.DEPOSITO);
        expect(movimiento.monto).toBe(5000);
        expect(movimiento.fecha).toEqual(fecha);
        expect(movimiento.descripcion).toBe("Depósito inicial");
        expect(movimiento.saldoPosterior).toBe(5000);
    });

    test("debe devolver una copia de la fecha", () => {
        const fecha = new Date("2026-07-28T15:00:00");

        const movimiento = new Movimiento(
            "MOV-1",
            TipoMovimiento.DEPOSITO,
```

```
        1000,
        fecha,
        "Depósito",
        1000
    );

    const fechaObtenida = movimiento.fecha;
    fechaObtenida.setFullYear(2030);

    expect(movimiento.fecha.getFullYear()).toBe(2026);
});

test("debe rechazar un identificador vacío", () => {
    expect(() => {
        new Movimiento(
            "",
            TipoMovimiento.DEPOSITO,
            1000,
            new Date(),
            "Depósito",
            1000
        );
    }).toThrow(
        "El identificador del movimiento es obligatorio."
    );
});

test("debe rechazar un tipo de movimiento inexistente", () => {
    expect(() => {
        new Movimiento(
            "MOV-1",
            "PAGO_DESCONOCIDO",
            1000,
            new Date(),
            "Operación",
            1000
        );
    }).toThrow("El tipo de movimiento no es válido.");
});

test("debe rechazar un monto igual a cero", () => {
    expect(() => {
        new Movimiento(
            "MOV-1",
            TipoMovimiento.DEPOSITO,
            0,
            new Date(),
            "Depósito",
            0
        );
    }).toThrow(
        "El monto del movimiento debe ser mayor que cero."
    );
});
```



```
});

test("debe rechazar un monto negativo", () => {
  expect(() => {
    new Movimiento(
      "MOV-1",
      TipoMovimiento.RETIRO,
      -500,
      new Date(),
      "Retiro",
      1000
    );
  }).toThrow(
    "El monto del movimiento debe ser mayor que cero."
  );
});

test("debe rechazar una fecha inválida", () => {
  expect(() => {
    new Movimiento(
      "MOV-1",
      TipoMovimiento.DEPOSITO,
      500,
      new Date("fecha-invalida"),
      "Depósito",
      500
    );
  }).toThrow("La fecha del movimiento no es válida.");
});

test("debe rechazar una descripción vacía", () => {
  expect(() => {
    new Movimiento(
      "MOV-1",
      TipoMovimiento.DEPOSITO,
      500,
      new Date(),
      "",
      500
    );
  }).toThrow(
    "La descripción del movimiento es obligatoria."
  );
});

test("debe rechazar un saldo posterior negativo", () => {
  expect(() => {
    new Movimiento(
      "MOV-1",
      TipoMovimiento.RETIRO,
      500,
      new Date(),
      "Retiro",
      500
    );
  });
});
```

```
        -100
    );
    }).toThrow(
        "El saldo posterior no puede ser negativo."
    );
});
});
```

14. Pruebas de caja negra de Cuenta

test/Cuenta.test.js

</> JavaScript

```
import { Cliente } from "../src/model/Cliente.js";
import { Cuenta } from "../src/model/Cuenta.js";
import { EstadoCuenta } from "../src/model/EstadoCuenta.js";
import { TipoMovimiento } from "../src/model/TipoMovimiento.js";

describe("Clase Cuenta", () => {

    let cliente;
    let cuenta;

    beforeEach(() => {
        cliente = new Cliente(
            "CLI-001",
            "Ana",
            "Pérez",
            "30111222",
            "ana@email.com",
            "clave123"
        );

        cuenta = new Cuenta("CTA-001", cliente, 10000);
    });

    test("debe crear una cuenta activa", () => {
        expect(cuenta.codigo).toBe("CTA-001");
        expect(cuenta.titular).toBe(cliente);
        expect(cuenta.saldo).toBe(10000);
        expect(cuenta.estado).toBe(EstadoCuenta.ACTIVA);
        expect(cuenta.estaActiva()).toBe(true);
    });

    test("debe registrar el saldo inicial como movimiento", () => {
        expect(cuenta.movimientos).toHaveLength(1);

        const movimiento = cuenta.movimientos[0];
```

```
    expect(movimiento.tipo).toBe(TipoMovimiento.DEPOSITO);
    expect(movimiento.monto).toBe(10000);
    expect(movimiento.descripcion).toBe("Saldo inicial");
    expect(movimiento.saldoPosterior).toBe(10000);
  });

  test("una cuenta con saldo inicial cero no debe registrar movimientos", () => {
    const cuentaSinSaldo = new Cuenta(
      "CTA-002",
      cliente,
      0
    );

    expect(cuentaSinSaldo.saldo).toBe(0);
    expect(cuentaSinSaldo.movimientos).toHaveLength(0);
  });

  test("debe realizar un depósito", () => {
    const movimiento = cuenta.depositar(
      2500,
      "Depósito por ventanilla"
    );

    expect(cuenta.saldo).toBe(12500);
    expect(movimiento.tipo).toBe(TipoMovimiento.DEPOSITO);
    expect(movimiento.monto).toBe(2500);
    expect(movimiento.saldoPosterior).toBe(12500);
  });

  test("debe realizar un retiro", () => {
    const movimiento = cuenta.retirar(
      3000,
      "Extracción"
    );

    expect(cuenta.saldo).toBe(7000);
    expect(movimiento.tipo).toBe(TipoMovimiento.RETIRO);
    expect(movimiento.monto).toBe(3000);
    expect(movimiento.saldoPosterior).toBe(7000);
  });

  test("debe rechazar un retiro superior al saldo", () => {
    expect(() => {
      cuenta.retirar(20000);
    }).toThrow("Saldo insuficiente.");

    expect(cuenta.saldo).toBe(10000);
  });

  test("debe rechazar un depósito de monto cero", () => {
    expect(() => {
      cuenta.depositar(0);
    }).toThrow("El monto debe ser mayor que cero.");
  });
```

```
});

test("debe rechazar un depósito negativo", () => {
  expect(() => {
    cuenta.depositar(-100);
  }).toThrow("El monto debe ser mayor que cero.");
});

test("debe rechazar un retiro de monto cero", () => {
  expect(() => {
    cuenta.retirar(0);
  }).toThrow("El monto debe ser mayor que cero.");
});

test("debe rechazar una descripción vacía", () => {
  expect(() => {
    cuenta.depositar(1000, " ");
  }).toThrow(
    "La descripción de la operación es obligatoria."
  );
});

test("debe rechazar una fecha inválida", () => {
  expect(() => {
    cuenta.depositar(
      1000,
      "Depósito",
      new Date("fecha-invalida")
    );
  }).toThrow("La fecha de la operación no es válida.");
});

test("debe devolver una copia del arreglo de movimientos", () => {
  const movimientosExternos = cuenta.movimientos;

  movimientosExternos.push("elemento externo");

  expect(cuenta.movimientos).toHaveLength(1);
});

test("debe bloquear una cuenta activa", () => {
  cuenta.bloquear();

  expect(cuenta.estado).toBe(EstadoCuenta.BLOQUEADA);
  expect(cuenta.estaActiva()).toBe(false);
});

test("debe impedir depósitos en una cuenta bloqueada", () => {
  cuenta.bloquear();

  expect(() => {
    cuenta.depositar(1000);
  }).toThrow("La cuenta se encuentra bloqueada.");
});
```

```
});

test("debe impedir retiros en una cuenta bloqueada", () => {
    cuenta.bloquear();

    expect(() => {
        cuenta.retirar(1000);
    }).toThrow("La cuenta se encuentra bloqueada.");
});

test("debe desbloquear una cuenta bloqueada", () => {
    cuenta.bloquear();
    cuenta.desbloquear();

    expect(cuenta.estado).toBe(EstadoCuenta.ACTIVA);
});

test("debe rechazar bloquear dos veces la misma cuenta", () => {
    cuenta.bloquear();

    expect(() => {
        cuenta.bloquear();
    }).toThrow("La cuenta ya se encuentra bloqueada.");
});

test("debe rechazar desbloquear una cuenta activa", () => {
    expect(() => {
        cuenta.desbloquear();
    }).toThrow("La cuenta no se encuentra bloqueada.");
});

test("debe cerrar una cuenta con saldo cero", () => {
    cuenta.retirar(10000);
    cuenta.cerrar();

    expect(cuenta.estado).toBe(EstadoCuenta.CERRADA);
    expect(cuenta.estaActiva()).toBe(false);
});

test("debe impedir cerrar una cuenta con saldo", () => {
    expect(() => {
        cuenta.cerrar();
    }).toThrow(
        "La cuenta solo puede cerrarse cuando su saldo es cero."
    );
});

test("debe impedir operaciones sobre una cuenta cerrada", () => {
    cuenta.retirar(10000);
    cuenta.cerrar();

    expect(() => {
        cuenta.depositar(1000);
    });
});
```

```

    }).toThrow("La cuenta se encuentra cerrada.");

    expect(() => {
      cuenta.retirar(1000);
    }).toThrow("La cuenta se encuentra cerrada.");
  });

  test("debe impedir cerrar dos veces la misma cuenta", () => {
    cuenta.retirar(10000);
    cuenta.cerrar();

    expect(() => {
      cuenta.cerrar();
    }).toThrow("La cuenta ya se encuentra cerrada.");
  });

  test("debe rechazar un código vacío", () => {
    expect(() => {
      new Cuenta("", cliente, 1000);
    }).toThrow("El código de la cuenta es obligatorio.");
  });

  test("debe rechazar un titular que no sea Cliente", () => {
    expect(() => {
      new Cuenta("CTA-002", {}, 1000);
    }).toThrow(
      "El titular de la cuenta debe ser un objeto Cliente."
    );
  });

  test("debe rechazar un saldo inicial negativo", () => {
    expect(() => {
      new Cuenta("CTA-002", cliente, -1000);
    }).toThrow(
      "El saldo inicial debe ser un número igual o mayor que cero."
    );
  });
});

```

15. Pruebas de caja negra de TransferenciaService

test/TransferenciaService.test.js

JavaScript

```

import { Cliente } from "../src/model/Cliente.js";
import { Cuenta } from "../src/model/Cuenta.js";
import { TipoMovimiento } from "../src/model/TipoMovimiento.js";
import { TransferenciaService } from "../src/service/TransferenciaService.js";

```

```
describe("TransferenciaService", () => {

    let clienteOrigen;
    let clienteDestino;
    let cuentaOrigen;
    let cuentaDestino;
    let transferenciaService;

    beforeEach(() => {
        clienteOrigen = new Cliente(
            "CLI-001",
            "Ana",
            "Pérez",
            "30111222",
            "ana@email.com",
            "clave123"
        );

        clienteDestino = new Cliente(
            "CLI-002",
            "Juan",
            "Gómez",
            "32123456",
            "juan@email.com",
            "clave456"
        );

        cuentaOrigen = new Cuenta(
            "CTA-001",
            clienteOrigen,
            10000
        );

        cuentaDestino = new Cuenta(
            "CTA-002",
            clienteDestino,
            2000
        );

        transferenciaService = new TransferenciaService();
    });

    test("debe transferir dinero entre dos cuentas", () => {
        const resultado = transferenciaService.transferir(
            cuentaOrigen,
            cuentaDestino,
            3000,
            "Pago de servicio"
        );

        expect(cuentaOrigen.saldo).toBe(7000);
        expect(cuentaDestino.saldo).toBe(5000);
    });
});
```

```
expect(resultado.movimientoOrigen.tipo).toBe(
  TipoMovimiento.TRANSFERENCIA_ENVIADA
);

expect(resultado.movimientoDestino.tipo).toBe(
  TipoMovimiento.TRANSFERENCIA_RECIBIDA
);

expect(resultado.movimientoOrigen.monto).toBe(3000);
expect(resultado.movimientoDestino.monto).toBe(3000);
});

test("debe registrar los movimientos en ambas cuentas", () => {
  transferenciaService.transferir(
    cuentaOrigen,
    cuentaDestino,
    1000
  );

  const movimientosOrigen = cuentaOrigen.movimientos;
  const movimientosDestino = cuentaDestino.movimientos;

  expect(movimientosOrigen).toHaveLength(2);
  expect(movimientosDestino).toHaveLength(2);

  expect(movimientosOrigen[1].tipo).toBe(
    TipoMovimiento.TRANSFERENCIA_ENVIADA
  );

  expect(movimientosDestino[1].tipo).toBe(
    TipoMovimiento.TRANSFERENCIA_RECIBIDA
  );
});

test("debe rechazar una transferencia sin saldo suficiente", () => {
  expect(() => {
    transferenciaService.transferir(
      cuentaOrigen,
      cuentaDestino,
      20000
    );
  }).toThrow(
    "La cuenta de origen no tiene saldo suficiente."
  );

  expect(cuentaOrigen.saldo).toBe(10000);
  expect(cuentaDestino.saldo).toBe(2000);
});

test("debe rechazar una transferencia de monto cero", () => {
  expect(() => {
    transferenciaService.transferir(
      cuentaOrigen,
```



```
        cuentaDestino,
        0
    );
}).toThrow(
    "El monto de la transferencia debe ser mayor que cero."
);
});

test("debe rechazar una transferencia negativa", () => {
    expect(() => {
        transferenciaService.transferir(
            cuentaOrigen,
            cuentaDestino,
            -500
        );
    }).toThrow(
        "El monto de la transferencia debe ser mayor que cero."
    );
});

test("debe rechazar una transferencia a la misma cuenta", () => {
    expect(() => {
        transferenciaService.transferir(
            cuentaOrigen,
            cuentaOrigen,
            1000
        );
    }).toThrow(
        "La cuenta de origen y destino deben ser diferentes."
    );
});

test("debe rechazar un origen que no sea Cuenta", () => {
    expect(() => {
        transferenciaService.transferir(
            {},
            cuentaDestino,
            1000
        );
    }).toThrow(
        "La cuenta de origen debe ser un objeto Cuenta."
    );
});

test("debe rechazar un destino que no sea Cuenta", () => {
    expect(() => {
        transferenciaService.transferir(
            cuentaOrigen,
            {},
            1000
        );
    }).toThrow(
        "La cuenta de destino debe ser un objeto Cuenta."
    );
});
```

```
    });  
});  
  
test("debe rechazar transferencias desde una cuenta bloqueada", () => {  
    cuentaOrigen.bloquear();  
  
    expect(() => {  
        transferenciaService.transferir(  
            cuentaOrigen,  
            cuentaDestino,  
            1000  
        );  
    }).toThrow(  
        "La cuenta de origen debe encontrarse activa."  
    );  
});  
  
test("debe rechazar transferencias hacia una cuenta bloqueada", () => {  
    cuentaDestino.bloquear();  
  
    expect(() => {  
        transferenciaService.transferir(  
            cuentaOrigen,  
            cuentaDestino,  
            1000  
        );  
    }).toThrow(  
        "La cuenta de destino debe encontrarse activa."  
    );  
});  
  
test("debe rechazar una descripción vacía", () => {  
    expect(() => {  
        transferenciaService.transferir(  
            cuentaOrigen,  
            cuentaDestino,  
            1000,  
            ""  
        );  
    }).toThrow(  
        "La descripción de la transferencia es obligatoria."  
    );  
});  
  
test("debe rechazar una fecha inválida", () => {  
    expect(() => {  
        transferenciaService.transferir(  
            cuentaOrigen,  
            cuentaDestino,  
            1000,  
            "Transferencia",  
            new Date("fecha-invalida")  
        );  
    });  
});
```

```
    }).toThrow(  
        "La fecha de la transferencia no es válida."  
    );  
});  
});
```

16. Aplicación de SRP

La propuesta distribuye las responsabilidades de la siguiente manera:

Clase	Responsabilidad principal
Cliente	Gestionar datos y credenciales del cliente
Cuenta	Gestionar saldo, estado e historial de una cuenta
Movimiento	Representar una operación histórica
TransferenciaService	Coordinar una operación entre dos cuentas
TipoMovimiento	Definir los tipos de movimientos permitidos
EstadoCuenta	Definir los estados posibles de una cuenta

Por ejemplo, `Cuenta` no valida contraseñas y `Cliente` no realiza depósitos. Del mismo modo, una transferencia no se ubica completamente dentro de una sola cuenta, porque necesita coordinar dos objetos.

17. Pruebas de caja negra

Las pruebas se consideran de caja negra porque verifican:

- Entradas válidas.
- Entradas inválidas.
- Resultados obtenidos.
- Excepciones esperadas.
- Cambios observables de estado.

Las pruebas no acceden a:

- Campos privados.
- Métodos privados.
- Secuencias internas de implementación.
- Estructuras internas modificables.

Por ejemplo, la prueba de un depósito verifica el nuevo saldo y el movimiento generado, pero no accede directamente a `#saldo` ni a `#movimientos`.

La siguiente etapa natural será incorporar las clases de repositorio para gestionar colecciones de clientes y cuentas, controlar identificadores únicos y separar el almacenamiento del modelo de dominio.



devolver un detalle de las relaciones entre clases y el tipo de relación

Detalle de las relaciones entre clases

1. `Cliente` — `Cuenta`

Tipo de relación: asociación.

 Mermaid

Diagrama inválido o no compatible.

Cada cuenta tiene exactamente un cliente titular:

 JavaScript

```
#titular;
```

El objeto `Cuenta` recibe un `Cliente` en su constructor:

 JavaScript

```
constructor(codigo, titular, saldoInicial = 0) {  
  this.#validarTitular(titular);  
  this.#titular = titular;  
}
```

La navegabilidad es desde **Cuenta** hacia **Cliente**, porque **Cuenta** conoce a su titular:

JavaScript

```
cuenta.titular
```

Sin embargo, **Cliente** no mantiene una colección de cuentas.

Cardinalidad

- Un **Cliente** puede estar relacionado con **0..*** cuentas.
- Una **Cuenta** debe tener exactamente **1** cliente.

¿Por qué es asociación?

Porque ambos objetos pueden existir de manera independiente:

- Un cliente puede existir aunque todavía no tenga cuentas.
- Una cuenta necesita un cliente para ser creada, pero la eliminación de la cuenta no implica eliminar al cliente.

No se considera composición porque el ciclo de vida del cliente no depende del ciclo de vida de la cuenta.

Tampoco es necesario modelarla como agregación, ya que **Cliente** no contiene una colección de cuentas. La relación se representa de forma más clara como una asociación navegable desde **Cuenta**.

2. Cuenta — Movimiento

Tipo de relación: composición.

Mermaid

Diagrama inválido o no compatible.

La composición se representa en UML con un rombo negro del lado de **Cuenta**.

La cuenta mantiene internamente sus movimientos:

```
</> JavaScript
```

```
#movimientos;
```

La colección se inicializa dentro de la propia cuenta:

```
</> JavaScript
```

```
this.#movimientos = [];
```

Los movimientos son creados por un método privado de **Cuenta**:

```
</> JavaScript
```

```
#registrarMovimiento(tipo, monto, descripcion, fecha) {  
  const movimiento = new Movimiento(  
    idMovimiento,  
    tipo,  
    monto,  
    fecha,  
    descripcion,  
    this.#saldo  
  );  
  
  this.#movimientos.push(movimiento);  
  
  return movimiento;  
}
```

Cardinalidad

- Una **Cuenta** puede contener **0..*** movimientos.
- Cada **Movimiento** pertenece conceptualmente a **1** cuenta.

¿Por qué es composición?

Porque el movimiento forma parte del historial interno de una cuenta.

Un movimiento como:

Retiro de \$500

Saldo posterior: \$4.500

no tiene significado completo sin la cuenta en la que ocurrió.

Además:

- La cuenta crea sus movimientos.
- La cuenta los almacena.
- Un movimiento no se comparte entre varias cuentas.
- El historial depende del ciclo de vida de la cuenta.

Por eso existe una relación fuerte de pertenencia.

3. TransferenciaService — Cuenta

Tipo de relación: dependencia.

Mermaid

Diagrama inválido o no compatible.

La dependencia se representa en UML mediante una línea discontinua con flecha.

`TransferenciaService` no almacena cuentas como atributos. Las recibe temporalmente como parámetros:

JavaScript

```
transferir(  
  cuentaOrigen,  
  cuentaDestino,  
  monto,  
  descripcion,  
  fecha  
) {  
  // ...  
}
```

Durante la operación invoca métodos públicos de ambas cuentas:

```
</> JavaScript
```

```
cuentaOrigen.retirar(...);  
cuentaDestino.depositar(...);
```

Cardinalidad

En una transferencia particular intervienen:

- 1 cuenta de origen.
- 1 cuenta de destino.

Sin embargo, en una dependencia normalmente no es imprescindible expresar cardinalidad, porque no representa una estructura permanente entre objetos.

¿Por qué es dependencia?

Porque `TransferenciaService` solamente utiliza objetos `Cuenta` durante la ejecución de un método.

No existe un atributo como:

```
</> JavaScript
```

```
#cuentaOrigen;  
#cuentaDestino;
```

Por lo tanto, las cuentas no quedan almacenadas dentro del servicio.

4. Cuenta — EstadoCuenta

Tipo de relación: dependencia o uso de tipo.

```
📄 Mermaid
```

Diagrama inválido o no compatible.

`Cuenta` utiliza los valores definidos en `EstadoCuenta` :


```
</> JavaScript
```

```
this.#estado = EstadoCuenta.ACTIVA;
```

También los consulta para validar operaciones:

```
</> JavaScript
```

```
if (this.#estado === EstadoCuenta.BLOQUEADA) {  
    throw new Error("La cuenta se encuentra bloqueada.");  
}
```

¿Por qué es dependencia?

Porque `EstadoCuenta` no forma parte del objeto como una entidad independiente. Es un conjunto de constantes que `Cuenta` utiliza para controlar sus posibles estados.

El atributo:

```
</> JavaScript
```

```
#estado;
```

almacena uno de los valores de la enumeración:

```
</> JavaScript
```

```
EstadoCuenta.ACTIVA  
EstadoCuenta.BLOQUEADA  
EstadoCuenta.CERRADA
```

5. `Cuenta` — `TipoMovimiento`

Tipo de relación: dependencia.

```
📄 Mermaid
```

Diagrama inválido o no compatible.

`Cuenta` utiliza `TipoMovimiento` cuando genera operaciones:

JavaScript

```
this.#registrarMovimiento(  
  TipoMovimiento.DEPOSITO,  
  monto,  
  descripcion,  
  fecha  
);
```

También valida qué tipos pueden utilizarse en depósitos y retiros:

JavaScript

```
const tiposPermitidos = [  
  TipoMovimiento.DEPOSITO,  
  TipoMovimiento.TRANSFERENCIA_RECIBIDA  
];
```

¿Por qué es dependencia?

Porque `Cuenta` no contiene un objeto `TipoMovimiento`. Solamente utiliza sus valores constantes para clasificar las operaciones.

6. `Movimiento` — `TipoMovimiento`

Tipo de relación: dependencia.

Mermaid

Diagrama inválido o no compatible.

La clase `Movimiento` utiliza la enumeración para validar el tipo recibido:

JavaScript

```
const tiposValidos = Object.values(TipoMovimiento);

if (!tiposValidos.includes(tipo)) {
  throw new Error("El tipo de movimiento no es válido.");
}
```

¿Por qué es dependencia?

Porque `Movimiento` necesita conocer los tipos permitidos para validar su constructor, pero no contiene ni administra un objeto independiente de tipo `TipoMovimiento`.

El atributo:

JavaScript

```
#tipo;
```

almacena simplemente un valor como:

JavaScript

```
"DEPOSITO"
```

O:

JavaScript

```
"RETIRO"
```

7. `TransferenciaService` — `TipoMovimiento`

Tipo de relación: dependencia.

Aunque puede omitirse en un diagrama simplificado, el servicio también utiliza los tipos de movimiento:

`</>` JavaScript

```
TipoMovimiento.TRANSFERENCIA_ENVIADA
TipoMovimiento.TRANSFERENCIA_RECIBIDA
```

La relación podría representarse así:

 Mermaid

Diagrama inválido o no compatible.

El servicio necesita estos valores para indicar qué clase de movimiento debe registrar cada cuenta.

Resumen general

Clase origen	Clase relacionada	Tipo de relación	Cardinalidad	Justificación
Cuenta	Cliente	Asociación	Cada cuenta tiene 1 cliente; un cliente puede tener 0..* cuentas	La cuenta mantiene referencia a su titular
Cuenta	Movimiento	Composición	Una cuenta contiene 0..* movimientos	Los movimientos forman parte del historial de la cuenta
TransferenciaServicio	Cuenta	Dependencia	Utiliza dos cuentas por transferencia	Las recibe como parámetros y no almacena
Cuenta	EstadoCuenta	Dependencia	No aplica cardinalidad estructural	Utiliza constantes para representar su estado

Clase origen	Clase relacionada	Tipo de relación	Cardinalidad	Justificación
Cuenta	TipoMovimiento	Dependencia	No aplica cardinalidad estructural	Utiliza constantes crear movimiento
Movimiento	TipoMovimiento	Dependencia	No aplica cardinalidad estructural	Lo utiliza para validar y clasificar la operación
TransferenciaService	TipoMovimiento	Dependencia	No aplica cardinalidad estructural	Define el movimiento enviado y recibido

Diagrama actualizado con todas las relaciones

 Mermaid

classDiagram

```

class Cliente {
    -String id
    -String nombre
    -String apellido
    -String dni
    -String email
    -String password
}

class Cuenta {
    -String codigo
    -Cliente titular
    -Number saldo
    -EstadoCuenta estado
    -Movimiento[] movimientos
    +depositar()
  
```

La relación estructural más fuerte del modelo es la **composición** entre **Cuenta** y **Movimiento**. La relación entre **Cuenta** y **Cliente** es una **asociación**, mientras que las restantes son principalmente **dependencias de uso**.

